

DV - Optimizing IT Support Team Performance Using Analytics (Supportlytics)

Milestone 1 - Week 1 & 2 Module 1

1. Objectives

This project analyzes IT support ticket data to uncover performance trends, optimize resolution times, and improve overall service efficiency. The analysis will translate raw ticket-level data into actionable operational recommendations for IT support leadership.

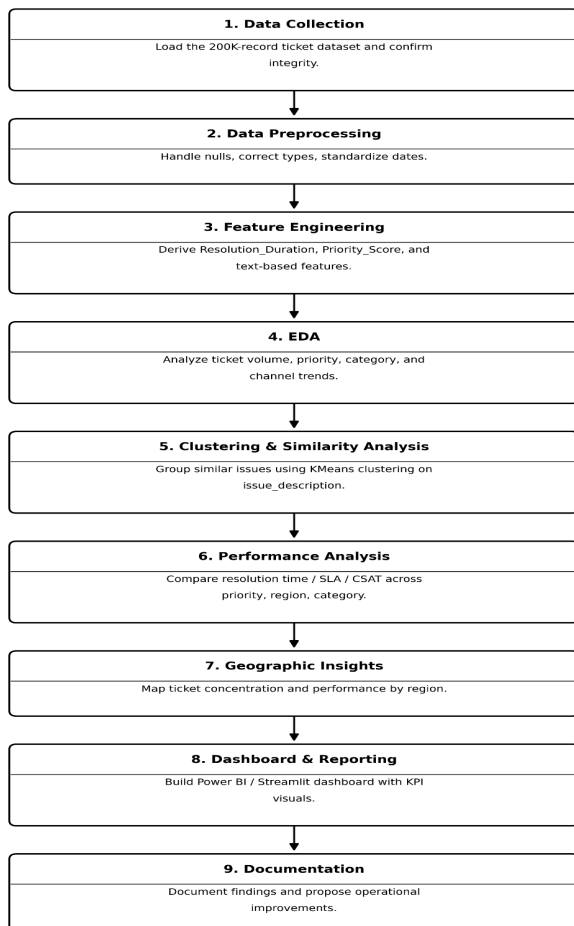
- Understand and process the IT support ticket dataset.
- Explore trends in ticket volume, priority, category, and channel distribution.
- Identify clusters of similar issues using text similarity and clustering techniques applied to issue descriptions.
- Quantify support performance through resolution time, first-response time, SLA compliance, and escalation behavior.
- Visualize performance metrics through dashboards and charts.
- Summarize the final analytical report and presentation.

2. KPIs and Workflow

2.1 Key Performance Indicators

- **Average Resolution Time:** mean time between ticket creation and resolution/closure
- **First Response Time:** average time to the first agent reply
- **Ticket Volume by Category/Type:** count of tickets per issue category (billing, technical, account, etc.)
- **Ticket Volume by Priority:** distribution across low/medium/high/urgent
- **Ticket Volume by Channel:** split across email, chat, phone, social media
- **SLA Compliance Rate:** % of tickets resolved within target time
- **Customer Satisfaction Score:** average rating if a satisfaction field exists
- **Backlog / Open Ticket Rate:** proportion of tickets still open/pending vs. closed

2.2 Workflow



3. Load the CSV Dataset Using Pandas

The dataset was loaded and using pandas to confirm structure.

Code:

```
import pandas as pd
df = pd.read_csv("./customer_support_tickets_200k.csv")
df.head()
```

	ticket_id	customer_name	customer_email	product	category	issue_description	resolution_notes	priority	status	channel	...
0	1	Patricia Smith	patricia.smith760@outlook.com	Web Portal	Account Suspension	The payment was deducted from my bank account ...	Data synchronization restored after backend se...	Urgent	Open	Email	...
1	2	Patricia Williams	patricia.williams390@gmail.com	Mobile App	Performance Issue	I found a bug in the latest update affecting r...	Provided step-by-step troubleshooting instruct...	Urgent	Closed	Email	...
2	3	William Anderson	william.anderson651@outlook.com	Web Portal	Performance Issue	The application crashes whenever I try to uplo...	Provided step-by-step troubleshooting instruct...	Medium	Closed	Chat	...
3	4	David Miller	david.miller672@icloud.com	Payment Gateway	Subscription Cancellation	My subscription was cancelled without my reque...	Provided step-by-step troubleshooting instruct...	Medium	Closed	Social Media	...
4	5	Robert Gonzalez	robert.gonzalez391@hotmail.com	Web Portal	Feature Request	The system is not syncing data across devices ...	We have reset the account credentials and advi...	High	Pending Customer	Email	...

5 rows × 30 columns

4. Explore schema & data types

Code:

```
df.info()
df['ticket_created_date'] = pd.to_datetime(df['ticket_created_date'])
df['ticket_resolved_date'] = pd.to_datetime(df['ticket_resolved_date'])
df[['ticket_created_date', 'ticket_resolved_date']].dtypes
```

Before Dtype Changed:

```
<class 'pandas.DataFrame'>
RangeIndex: 200000 entries, 0 to 199999
Data columns (total 30 columns):
#   Column                Non-Null Count  Dtype
---  -
0   ticket_id              200000 non-null  int64
1   customer_name          200000 non-null  str
2   customer_email         200000 non-null  str
3   product                200000 non-null  str
4   category               200000 non-null  str
5   issue_description      200000 non-null  str
6   resolution_notes       200000 non-null  str
7   priority               200000 non-null  str
8   status                 200000 non-null  str
9   channel                200000 non-null  str
10  region                 200000 non-null  str
11  customer_age           200000 non-null  int64
12  customer_gender        200000 non-null  str
13  subscription_type      200000 non-null  str
14  customer_tenure_months 200000 non-null  int64
15  previous_tickets       200000 non-null  int64
16  customer_satisfaction_score 200000 non-null  int64
17  first_response_time_hours 200000 non-null  float64
18  resolution_time_hours  200000 non-null  float64
19  ticket_created_date    200000 non-null  str
...
28  issue_complexity_score 200000 non-null  int64
29  customer_segment       200000 non-null  str
dtypes: float64(2), int64(6), str(22)
memory usage: 45.8 MB
```

After Dtype Changed:

```
ticket_id              int64
customer_name          str
customer_email         str
product                str
category               str
issue_description      str
resolution_notes       str
priority               str
status                 str
channel                str
region                 str
customer_age           int64
customer_gender        str
subscription_type      str
customer_tenure_months int64
previous_tickets       int64
customer_satisfaction_score int64
first_response_time_hours float64
resolution_time_hours  float64
ticket_created_date    datetime64[us]
ticket_resolved_date   datetime64[us]
escalated              str
sla_breached           str
operating_system       str
browser                str
payment_method         str
language               str
preferred_contact_time str
issue_complexity_score int64
customer_segment       str
```

5. missing values

Code:

```
df.isnull().sum()
```

```
ticket_id              0
customer_name          0
customer_email         0
product                0
category               0
issue_description      0
resolution_notes       0
priority               0
status                 0
channel                0
region                 0
customer_age           0
customer_gender        0
subscription_type      0
customer_tenure_months 0
previous_tickets       0
customer_satisfaction_score 0
first_response_time_hours 0
resolution_time_hours  0
ticket_created_date    0
ticket_resolved_date   0
escalated              0
sla_breached           0
operating_system       0
browser                40023
...
language               0
preferred_contact_time 0
issue_complexity_score 0
customer_segment       0
dtype: int64
```

6. Calculate initial ticket distribution by Type, Priority, and Category

Note: The values are generated after the dataset is cleaned.

6.1 Ticket volume by Type:

Code:

```
print('--- Distribution by Type ---')
display(df['type'].value_counts().rename_axis('Type').reset_index(name='Count'))
```

	Type	Count
0	Request	80029
1	Problem	60052
2	Incident	59919

6.2 Ticket volume by Priority:

Code:

```
print('--- Distribution by Priority ---')
display(df['priority'].value_counts().rename_axis('Priority').reset_index(name='Count'))
```

	Priority	Count
0	High	50241
1	Urgent	50143
2	Medium	49854
3	Low	49762

6.2 Ticket volume by Category:

Code:

```
print('--- Distribution by Category ---')
display(df['category'].value_counts().rename_axis('Category').reset_index(name='Count'))
```

	Category	Count
0	Feature Request	20169
1	Subscription Cancellation	20096
2	Performance Issue	20074
3	Security Concern	20040
4	Login Issue	20002
5	Payment Problem	19997
6	Bug Report	19981
7	Refund Request	19900
8	Data Sync Issue	19877
9	Account Suspension	19864

Module 2

1. Handle missing or incorrect data in text fields

Code:

```
df.drop(columns=['browser'], inplace=True)
print('browser' in df.columns)
print(df.shape)
```

```
False
(200000, 29)
```

2. Create new features: Type

Code:

```
category_type_map = {
    'Account Suspension': 'Request',
    'Bug Report': 'Problem',
    'Data Sync Issue': 'Incident',
    'Feature Request': 'Request',
    'Login Issue': 'Incident',
    'Payment Problem': 'Problem',
    'Performance Issue': 'Problem',
    'Refund Request': 'Request',
    'Security Concern': 'Incident',
    'Subscription Cancellation': 'Request'
}
cat_idx = df.columns.get_loc('category')
df.insert(cat_idx + 1, 'type', df['category'].map(category_type_map))
df[['category', 'type']].head(10)
```

	category	type
0	Account Suspension	Request
1	Performance Issue	Problem
2	Performance Issue	Problem
3	Subscription Cancellation	Request
4	Feature Request	Request
5	Payment Problem	Problem
6	Security Concern	Incident
7	Subscription Cancellation	Request
8	Security Concern	Incident
9	Performance Issue	Problem

3. Create new features: Resolution_Duration and Priority_Score

Code:

```
df['ticket_created_date'] = pd.to_datetime(df['ticket_created_date'])
df['ticket_resolved_date'] = pd.to_datetime(df['ticket_resolved_date'])
df['resolution_duration'] = (df['ticket_resolved_date'] - df['ticket_created_date']).dt.days
```

```
priority_score_map = {'Low': 1, 'Medium': 2, 'High': 3, 'Urgent': 4}
df['priority_score'] = df['priority'].map(priority_score_map)
```

```
df[['priority', 'priority_score', 'ticket_created_date', 'ticket_resolved_date', 'resolution_duration']].head(10)
```

	priority	priority_score	ticket_created_date	ticket_resolved_date	resolution_duration
0	Urgent	4	2023-05-17	2023-05-20	3
1	Urgent	4	2024-01-06	2024-01-19	13
2	Medium	2	2022-11-30	2022-12-05	5
3	Medium	2	2024-03-21	2024-04-04	14
4	High	3	2024-08-16	2024-08-24	8
5	Low	1	2024-09-27	2024-09-27	0
6	Medium	2	2024-02-04	2024-02-11	7
7	Low	1	2024-08-26	2024-08-29	3
8	Low	1	2024-06-26	2024-07-08	12
9	Urgent	4	2022-05-05	2022-05-16	11

4. Create new features: Queue

```
queue_map = {
    'Account Suspension': 'Account Management',
    'Bug Report': 'Development Team',
    'Data Sync Issue': 'Technical Support',
    'Feature Request': 'Product Team',
    'Login Issue': 'Technical Support',
```

```

    'Payment Problem': 'Billing Support',
    'Performance Issue': 'Infrastructure Team',
    'Refund Request': 'Billing Support',
    'Security Concern': 'Security Team',
    'Subscription Cancellation': 'Account Management'
}
df['queue'] = df['category'].map(queue_map)
df[['category', 'queue']].head(10)

```

	category	queue
0	Account Suspension	Account Management
1	Performance Issue	Infrastructure Team
2	Performance Issue	Infrastructure Team
3	Subscription Cancellation	Account Management
4	Feature Request	Product Team
5	Payment Problem	Billing Support
6	Security Concern	Security Team
7	Subscription Cancellation	Account Management
8	Security Concern	Security Team
9	Performance Issue	Infrastructure Team

5. Exported the cleaned dataset

Code:

```

df.to_csv('./cleaned_customer_support_tickets_200k.csv', index=False)
print('Exported successfully:', df.shape)

```

```
Exported successfully: (200000, 32)
```

Milestone 2 - Week 3 & 4

Module 3

Exploratory Visualization

- Plot ticket type distribution
- Identify top categories and clusters by frequency
- Visualize tickets by priority and queue

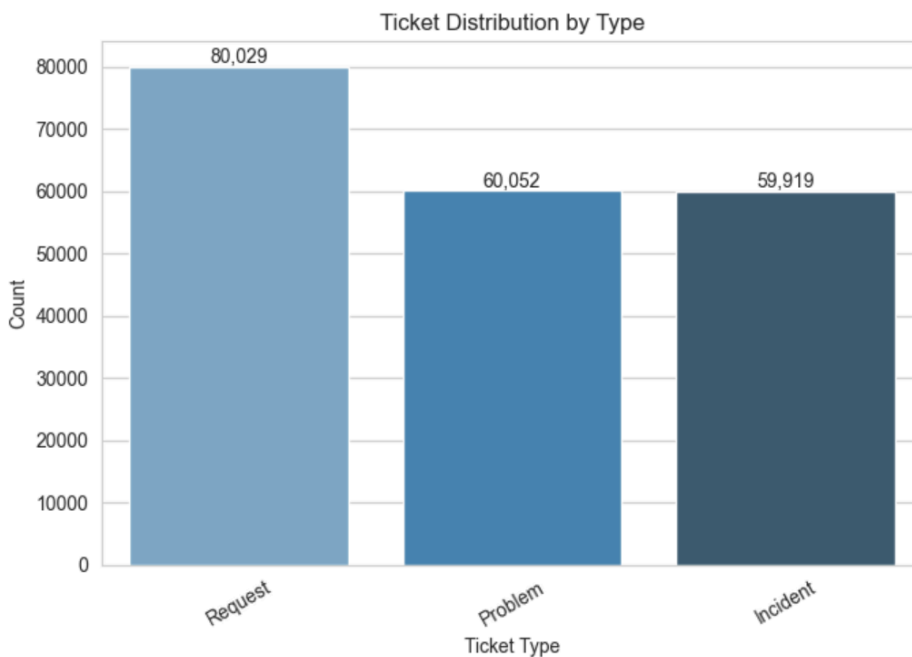
Code:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_style("whitegrid")
df = pd.read_csv("./cleaned_customer_support_tickets_200k.csv")
```

1. Ticket Type Distribution

```
plt.figure(figsize=(7,5))
ax = sns.countplot(data=df, x="type", order=df["type"].value_counts().index, palette="Blues_d")
plt.title("Ticket Distribution by Type")
plt.xlabel("Ticket Type")
plt.ylabel("Count")
plt.xticks(rotation=30)
for container in ax.containers:
    ax.bar_label(container, fmt=lambda x: f'{int(x):,}')
plt.tight_layout()
plt.savefig("./images/M3/plot_ticket_type_distribution.png", dpi=200)
plt.show()
```



2. Top Issue Categories by Frequency

```
plt.figure(figsize=(8,5))
top_categories = df["category"].value_counts().head(10)
ax = sns.barplot(x=top_categories.values, y=top_categories.index, palette="viridis")
plt.title("Top 10 Issue Categories by Frequency")
plt.xlabel("Number of Tickets")
plt.ylabel("Category")
```

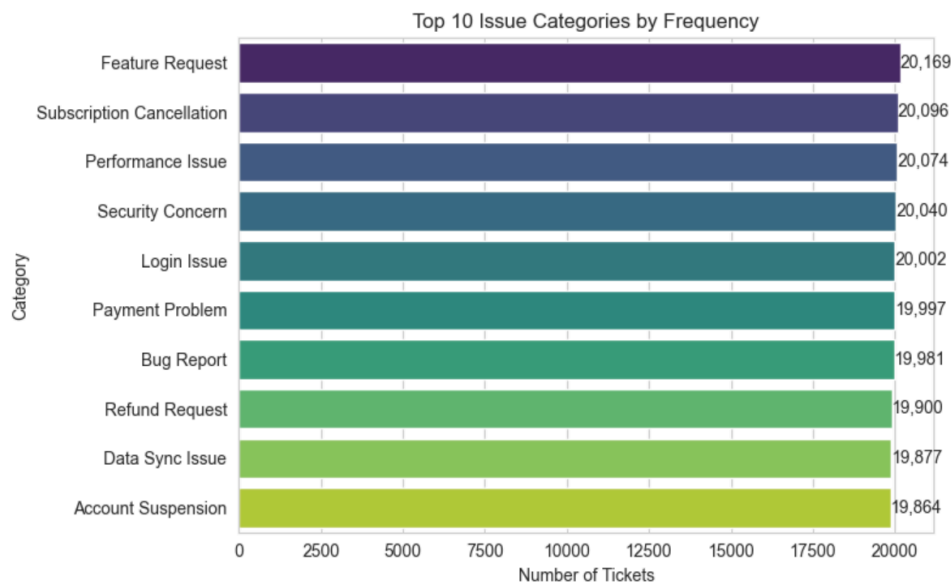
for container in ax.containers:

```
ax.bar_label(container, fmt=lambda x: f'{int(x):,}')
```

plt.tight_layout()

plt.savefig("./images/M3/plot_top_categories.png", dpi=200)

plt.show()



3. Tickets by Priority

```
plt.figure(figsize=(7,5))
```

```
ax = sns.countplot(data=df, x="priority", order=df["priority"].value_counts().index, palette="Reds_d")
```

```
plt.title("Ticket Volume by Priority")
```

```
plt.xlabel("Priority")
```

```
plt.ylabel("Count")
```

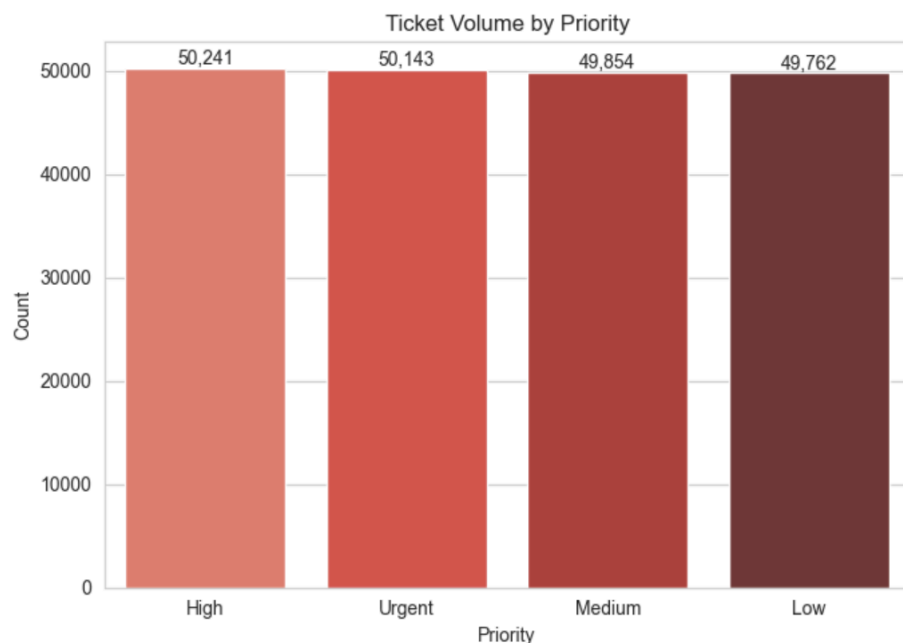
for container in ax.containers:

```
ax.bar_label(container, fmt=lambda x: f'{int(x):,}')
```

plt.tight_layout()

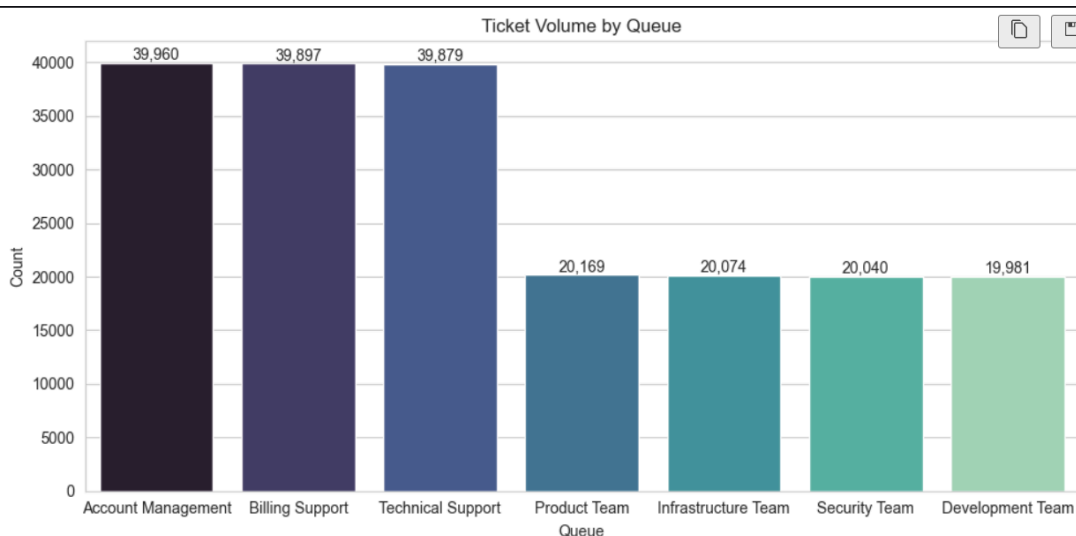
plt.savefig("./images/M3/plot_priority_distribution.png", dpi=200)

plt.show()



4. Tickets by Queue

```
plt.figure(figsize=(10,5))
ax = sns.countplot(data=df, x="queue", order=df["queue"].value_counts().index, palette="mako")
plt.title("Ticket Volume by Queue")
plt.xlabel("Queue")
plt.ylabel("Count")
for container in ax.containers:
    ax.bar_label(container, fmt=lambda x: f'{int(x):,}')
plt.tight_layout()
plt.savefig("./images/M3/plot_queue_distribution.png", dpi=200)
plt.show()
```



Module 4

Similarity and Cluster Insights

- Analyze average similarity scores within clusters
- Compare cluster size vs. issue type
- Visualize performance gaps using boxplots and scatter plots
- Deliverables: Minimum 8 visualizations, Summary of key patterns and problem clusters

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.decomposition import PCA

os.makedirs('images/M4', exist_ok=True)
sns.set_theme(style='whitegrid')
PALETTE = 'tab10'
N_CLUSTERS = 4
```


1. Load Data & Build Clusters

```
df = pd.read_csv('cleaned_customer_support_tickets_200k.csv')
df['issue_description'] = df['issue_description'].fillna("")

# TF-IDF vectorization
tfidf = TfidfVectorizer(max_features=500, stop_words='english')
X = tfidf.fit_transform(df['issue_description'])

# KMeans clustering into 4 clusters
km = KMeans(n_clusters=N_CLUSTERS, random_state=42, n_init=10)
df['cluster'] = km.fit_predict(X)
df['cluster_label'] = df['cluster'].map(lambda c: f'Cluster {c}')

print(df['cluster_label'].value_counts())
```

```
cluster_label
Cluster 0    99695
Cluster 1    40183
Cluster 3    39890
Cluster 2    20232
Name: count, dtype: int64
```

2. Compute Cosine Similarity Scores

```
# Sample 5000 rows for similarity computation (memory-safe)
sample = df.sample(5000, random_state=42).copy()
X_sample = tfidf.transform(sample['issue_description'])

# Cosine similarity of each doc to its cluster centroid
centroids = km.cluster_centers_
sim_scores = []
for i, row in enumerate(X_sample):
    c = sample.iloc[i]['cluster']
    score = cosine_similarity(row, centroids[c].reshape(1, -1))[0][0]
    sim_scores.append(score)

sample['similarity_score'] = sim_scores

# Merge similarity scores back to full df (NaN for non-sampled rows)
df = df.merge(sample[['ticket_id', 'similarity_score']], on='ticket_id', how='left')

avg_sim = sample.groupby('cluster_label')['similarity_score'].mean().reset_index()
print(avg_sim)
```

```
cluster_label  similarity_score
0      Cluster 0           0.447201
1      Cluster 1           0.742350
2      Cluster 2           1.000000
3      Cluster 3           0.753322
```

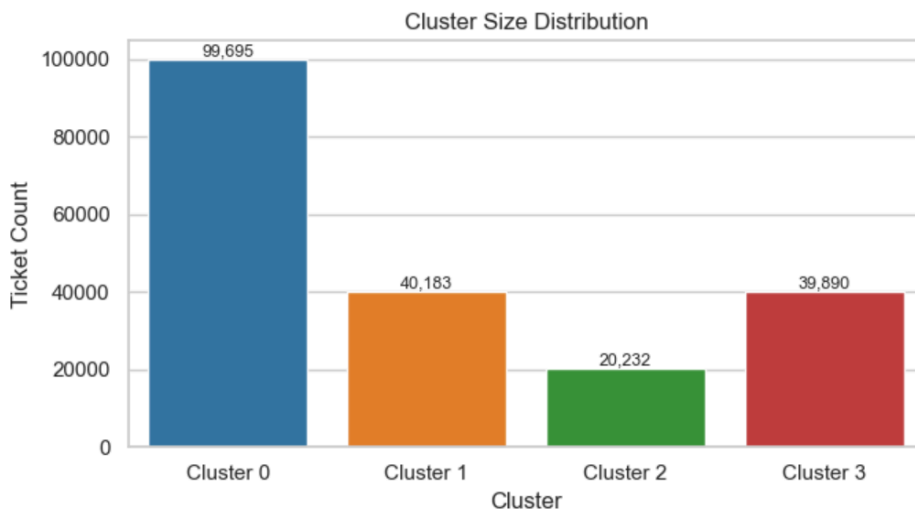
Plot 1 – Cluster Size Distribution (Bar)

```
fig, ax = plt.subplots(figsize=(7, 4))
counts = df['cluster_label'].value_counts().sort_index()
sns.barplot(x=counts.index, y=counts.values, palette=PALETTE, ax=ax)
ax.set_title('Cluster Size Distribution')
ax.set_xlabel('Cluster'); ax.set_ylabel('Ticket Count')
for p in ax.patches:
```

```

ax.annotate(f'{int(p.get_height()):,}', (p.get_x() + p.get_width()/2, p.get_height()),
            ha='center', va='bottom', fontsize=9)
plt.tight_layout()
plt.savefig('images/M4/plot1_cluster_size.png', dpi=150)
plt.show()

```

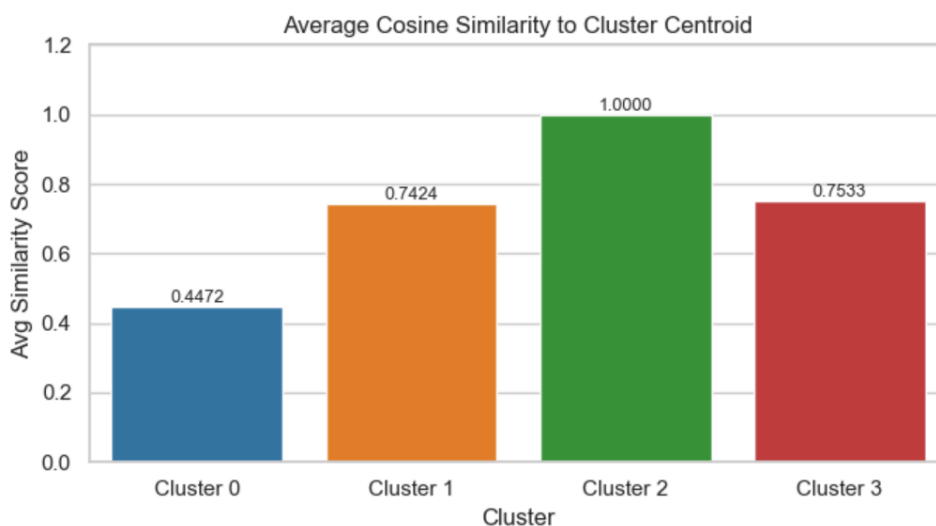


Plot 2 – Average Similarity Score per Cluster (Bar)

```

fig, ax = plt.subplots(figsize=(7, 4))
sns.barplot(data=avg_sim, x='cluster_label', y='similarity_score', palette=PALETTE, ax=ax)
ax.set_title('Average Cosine Similarity to Cluster Centroid')
ax.set_xlabel('Cluster'); ax.set_ylabel('Avg Similarity Score')
ax.set_ylim(0, avg_sim['similarity_score'].max() * 1.2)
for p in ax.patches:
    ax.annotate(f'{p.get_height():.4f}', (p.get_x() + p.get_width()/2, p.get_height()),
                ha='center', va='bottom', fontsize=9)
plt.tight_layout()
plt.savefig('images/M4/plot2_avg_similarity.png', dpi=150)
plt.show()

```



Plot 3 – Cluster vs Issue Type (Stacked Bar)

```

ct = df.groupby(['cluster_label', 'type']).size().unstack(fill_value=0)
ct_pct = ct.div(ct.sum(axis=1), axis=0) * 100

```

```

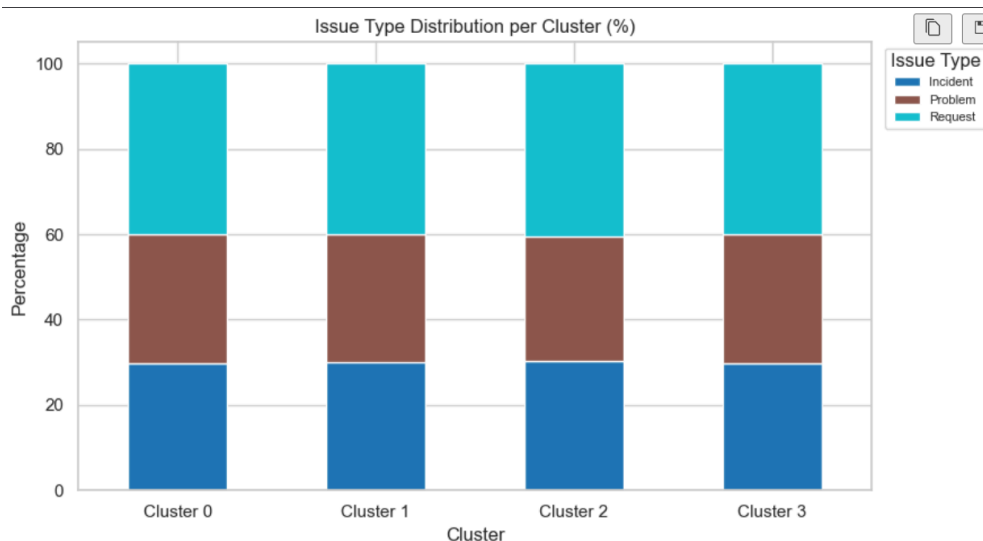
fig, ax = plt.subplots(figsize=(9, 5))
ct_pct.plot(kind='bar', stacked=True, colormap='tab10', ax=ax)
ax.set_title('Issue Type Distribution per Cluster (%)')
ax.set_xlabel('Cluster'); ax.set_ylabel('Percentage')

```

```

ax.legend(title='Issue Type', bbox_to_anchor=(1.01, 1), loc='upper left', fontsize=8)
plt.xticks(rotation=0)
plt.tight_layout()
plt.savefig('images/M4/plot3_cluster_issue_type.png', dpi=150)
plt.show()

```

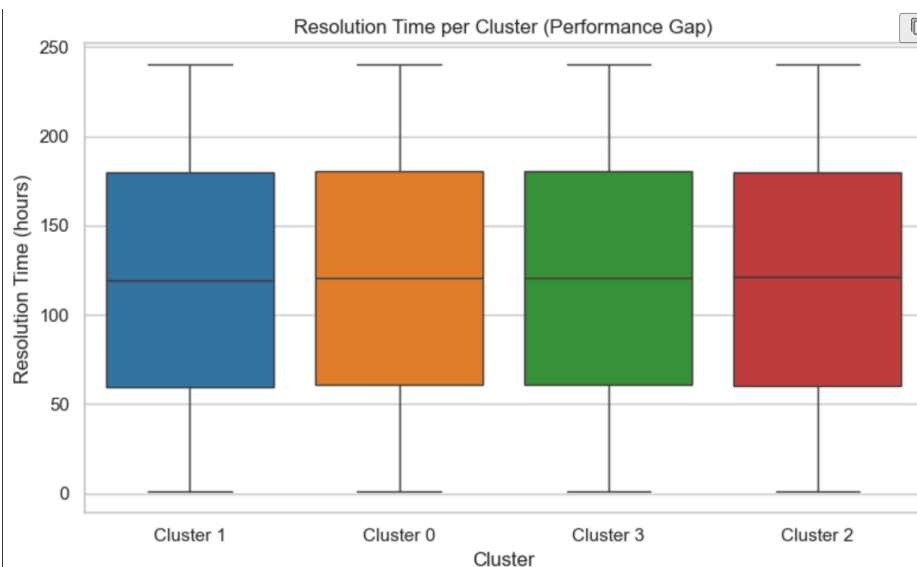


Plot 4 – Resolution Time Boxplot per Cluster (Performance Gap)

```

fig, ax = plt.subplots(figsize=(8, 5))
sns.boxplot(data=df, x='cluster_label', y='resolution_time_hours',
            palette=PALETTE, showfliers=False, ax=ax)
ax.set_title('Resolution Time per Cluster (Performance Gap)')
ax.set_xlabel('Cluster'); ax.set_ylabel('Resolution Time (hours)')
plt.tight_layout()
plt.savefig('images/M4/plot4_resolution_time_boxplot.png', dpi=150)
plt.show()

```

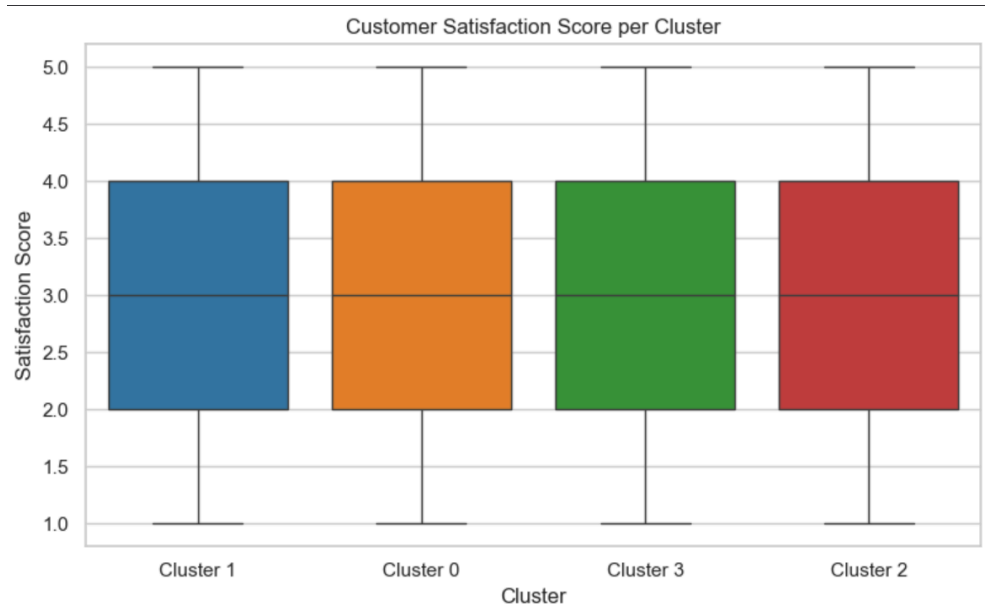


Plot 5 – Customer Satisfaction Score Boxplot per Cluster

```

fig, ax = plt.subplots(figsize=(8, 5))
sns.boxplot(data=df, x='cluster_label', y='customer_satisfaction_score',
            palette=PALETTE, ax=ax)
ax.set_title('Customer Satisfaction Score per Cluster')
ax.set_xlabel('Cluster'); ax.set_ylabel('Satisfaction Score')
plt.tight_layout()
plt.savefig('images/M4/plot5_satisfaction_boxplot.png', dpi=150)
plt.show()

```



Milestone 3 - Week 5 & 6

Module 5

Performance Trend Analysis

- Analyze average resolution times by priority and type
- Determine which countries or teams handle tickets fastest
- Identify high-priority unresolved issues

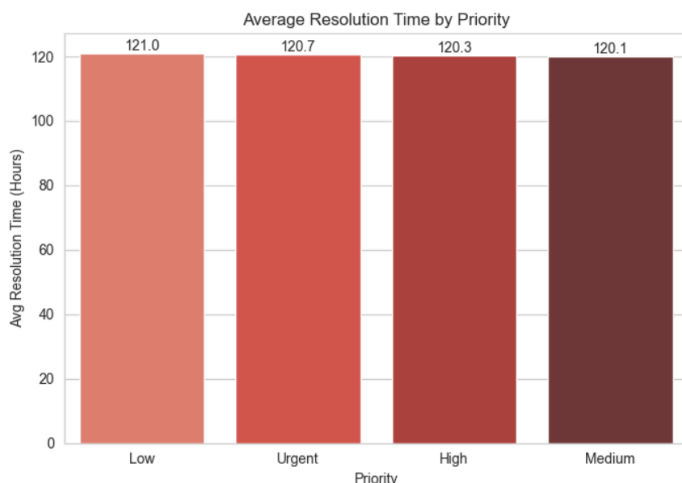
Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os
sns.set_style("whitegrid")
df = pd.read_csv("./cleaned_customer_support_tickets_200k.csv")
```

1. Average Resolution Time by Priority and Type

```
plt.figure(figsize=(7,5))
avg_by_priority = df.groupby("priority")["resolution_time_hours"].mean().sort_values(ascending=False)
ax = sns.barplot(x=avg_by_priority.index, y=avg_by_priority.values, palette="Reds_d")
plt.title("Average Resolution Time by Priority")
plt.xlabel("Priority")
plt.ylabel("Avg Resolution Time (Hours)")
for container in ax.containers:
    ax.bar_label(container, fmt=lambda x: f"{x:.1f}")
plt.tight_layout()
plt.savefig("./images/M5/plot_avg_resolution_by_priority.png", dpi=200)
plt.show()
```

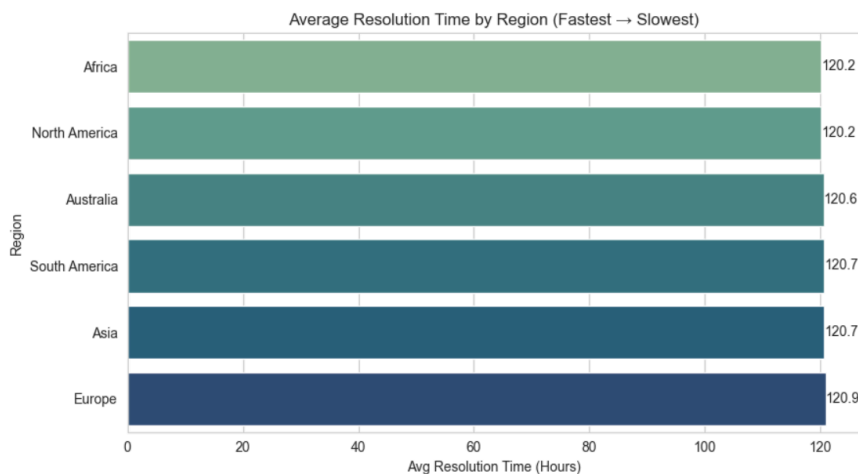
```
plt.figure(figsize=(7,5))
avg_by_type = df.groupby("type")["resolution_time_hours"].mean().sort_values(ascending=False)
ax = sns.barplot(x=avg_by_type.index, y=avg_by_type.values, palette="Blues_d")
plt.title("Average Resolution Time by Type")
plt.xlabel("Type")
plt.ylabel("Avg Resolution Time (Hours)")
for container in ax.containers:
    ax.bar_label(container, fmt=lambda x: f"{x:.1f}")
plt.tight_layout()
plt.savefig("./images/M5/plot_avg_resolution_by_type.png", dpi=200)
plt.show()
```



2. Fastest Regions ("Countries") and Teams ("Queues")

```
plt.figure(figsize=(9,5))
avg_by_region = df.groupby("region")["resolution_time_hours"].mean().sort_values()
ax = sns.barplot(x=avg_by_region.values, y=avg_by_region.index, palette="crest")
plt.title("Average Resolution Time by Region (Fastest → Slowest)")
plt.xlabel("Avg Resolution Time (Hours)")
plt.ylabel("Region")
for container in ax.containers:
    ax.bar_label(container, fmt=lambda x: f"{x:.1f}")
plt.tight_layout()
plt.savefig("./images/M5/plot_avg_resolution_by_region.png", dpi=200)
plt.show()
```

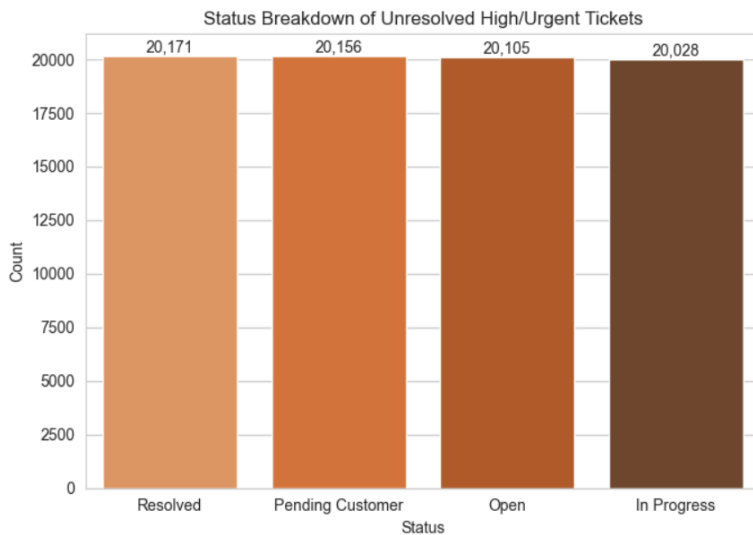
```
plt.figure(figsize=(9,5))
avg_by_queue = df.groupby("queue")["resolution_time_hours"].mean().sort_values()
ax = sns.barplot(x=avg_by_queue.values, y=avg_by_queue.index, palette="mako")
plt.title("Average Resolution Time by Team/Queue (Fastest → Slowest)")
plt.xlabel("Avg Resolution Time (Hours)")
plt.ylabel("Queue")
for container in ax.containers:
    ax.bar_label(container, fmt=lambda x: f"{x:.1f}")
plt.tight_layout()
plt.savefig("./images/M5/plot_avg_resolution_by_queue.png", dpi=200)
plt.show()
```



3. High-Priority Unresolved Issues

```
unresolved_high_priority = df[
    (df["priority"].isin(["High", "Urgent"])) &
    (df["status"] != "Closed")
]
print(f"High/Urgent priority tickets still unresolved: {len(unresolved_high_priority):,}")

plt.figure(figsize=(7,5))
status_breakdown = unresolved_high_priority["status"].value_counts()
ax = sns.barplot(x=status_breakdown.index, y=status_breakdown.values, palette="Oranges_d")
plt.title("Status Breakdown of Unresolved High/Urgent Tickets")
plt.xlabel("Status")
plt.ylabel("Count")
for container in ax.containers:
    ax.bar_label(container, fmt=lambda x: f'{int(x):,}')
plt.tight_layout()
plt.savefig("./images/M5/plot_unresolved_high_priority.png", dpi=200)
plt.show()
```



Module 6

Geographic and Category-Level Insights

- Create heatmaps of ticket concentration by region
- Map issue categories geographically using latitude/longitude
- Evaluate the relationship between cluster size and performance score
- Deliverables: Performance metrics summary, Geo-visualizations and comparative charts

Code:

```
sns.set_style("whitegrid")
os.makedirs("images/M6", exist_ok=True)
```

STEP 0: Confirm what 'region' actually contains

```
print("Unique regions:", df["region"].unique())
print("Number of regions:", df["region"].nunique())
```

```
Unique regions: <ArrowStringArray>
['North America', 'South America', 'Europe', 'Asia', 'Africa', 'Australia']
Length: 6, dtype: str
Number of regions: 6
```

1. Heatmap: Ticket Concentration by Region x Category

```
plt.figure(figsize=(12, 8))
region_category = pd.crosstab(df["region"], df["category"])
sns.heatmap(region_category, cmap="YlGnBu", annot=True, fmt="d", linewidths=0.5)
plt.title("Ticket Concentration by Region and Category")
plt.xlabel("Category")
plt.ylabel("Region")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.savefig("images/M6/plot1_region_category_heatmap.png", dpi=150)
plt.show()
```



2. Geographic Mapping — region -> country centroid lookup

```
import plotly.express as px
```

```
continent_map = {
    "Asia": "India",
    "Europe": "Germany",
    "North America": "United States",
    "South America": "Brazil",
    "Africa": "South Africa",
    "Australia": "Australia",
    "Oceania": "Australia"
}

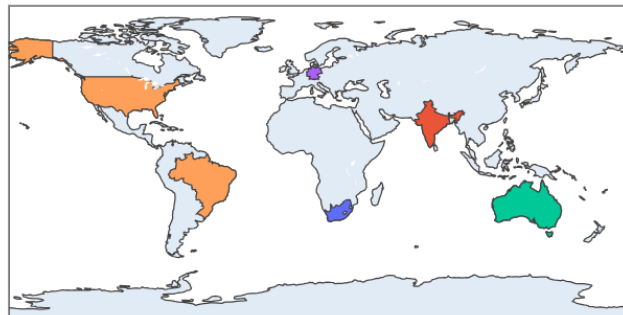
region_summary = (
    df.groupby("region")
    .agg(
        ticket_count=("ticket_id", "count"),
        top_category=("category", lambda x: x.mode()[0])
    )
    .reset_index()
```


)

```
region_summary["country"] = region_summary["region"].map(continent_map)
```

```
fig = px.choropleth(
    region_summary,
    locations="country",
    locationmode="country names",
    color="top_category",
    hover_name="region",
    hover_data=["ticket_count"],
    title="Dominant Ticket Category by Region"
)
fig.write_html("images/M6/plot2_geo_category_map.html")
fig.show()
```

Dominant Ticket Category by Region



top_category

- Performance Issue
- Data Sync Issue
- Login Issue
- Refund Request
- Subscription Cancellation

3. Cluster Size vs Performance Score

```
clustered = pd.read_csv("clustered_tickets.csv") # from Module 4 output
```

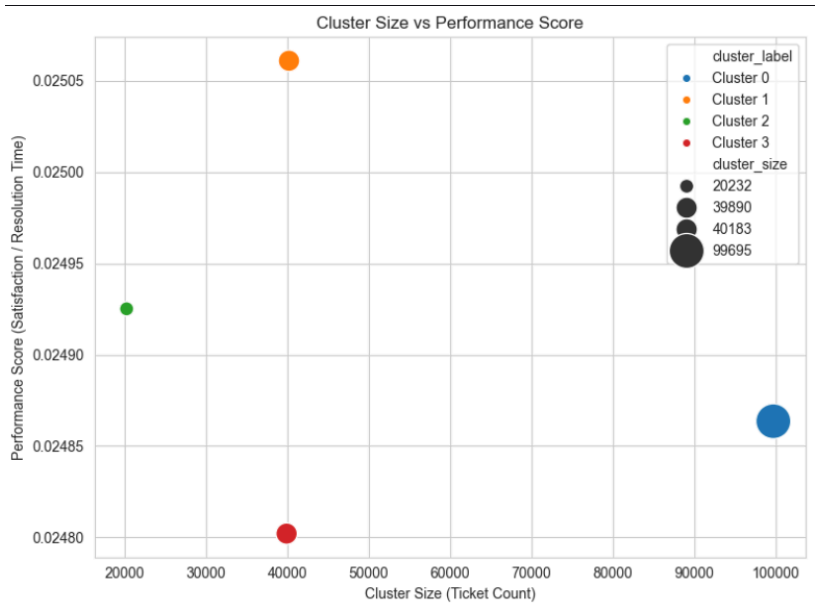
```
cluster_summary = clustered.groupby("cluster_label").agg(
    cluster_size=("ticket_id", "count"),
    avg_resolution_time=("resolution_time_hours", "mean"),
    avg_satisfaction=("customer_satisfaction_score", "mean"),
    avg_similarity=("similarity_score", "mean"),
).reset_index()

cluster_summary["performance_score"] = (
    cluster_summary["avg_satisfaction"] / cluster_summary["avg_resolution_time"]
)
```

```
plt.figure(figsize=(8, 6))
sns.scatterplot(
    data=cluster_summary, x="cluster_size", y="performance_score",
    size="cluster_size", hue="cluster_label", palette="tab10",
    legend="full", sizes=(100, 600)
)
plt.title("Cluster Size vs Performance Score")
plt.xlabel("Cluster Size (Ticket Count)")
plt.ylabel("Performance Score (Satisfaction / Resolution Time)")
plt.tight_layout()
plt.savefig("images/M6/plot3_cluster_size_vs_performance.png", dpi=150)
```

plt.show()

print(cluster_summary)



	cluster_label	cluster_size	avg_resolution_time	avg_satisfaction	\
0	Cluster 0	99695	120.753885	3.002377	
1	Cluster 1	40183	119.705125	2.999925	
2	Cluster 2	20232	120.475418	3.002867	
3	Cluster 3	39890	120.877797	2.998020	

	avg_similarity	performance_score
0	0.447201	0.024864
1	0.742350	0.025061
2	1.000000	0.024925
3	0.753322	0.024802

4. Performance Metrics Summary (deliverable)

```
performance_summary = df.groupby("region").agg(  
    ticket_count=("ticket_id", "count"),  
    avg_resolution_time=("resolution_time_hours", "mean"),  
    avg_first_response=("first_response_time_hours", "mean"),  
    avg_satisfaction=("customer_satisfaction_score", "mean"),  
)  
.sort_values("avg_resolution_time")  
  
performance_summary.to_csv("images/M6/performance_metrics_summary.csv")  
print(performance_summary)
```

	ticket_count	avg_resolution_time	avg_first_response	\
region				
Africa	33576	120.206789	36.408282	
North America	33177	120.208662	36.381035	
Australia	33047	120.576541	36.430527	
South America	33369	120.660512	36.214094	
Asia	33554	120.666593	36.174085	
Europe	33277	120.920070	36.248607	

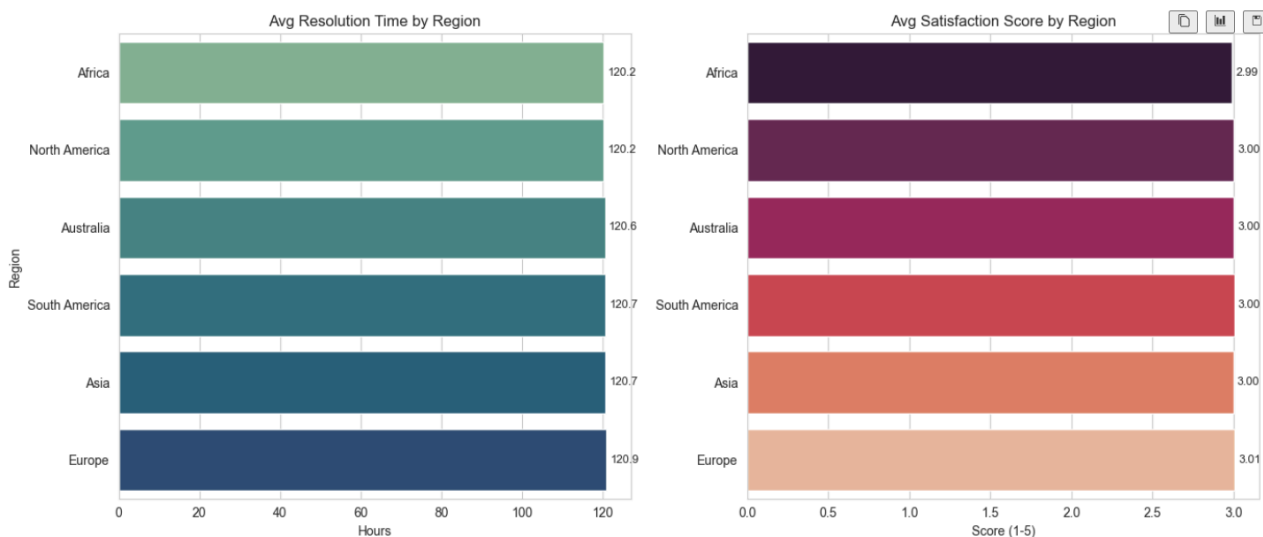
	avg_satisfaction
region	
Africa	2.990410
North America	3.001839
Australia	3.002965
South America	3.004465
Asia	2.998420
Europe	3.008414

5. Comparative Charts — Region Performance Comparison

```
plt.figure(figsize=(10, 6))
region_perf = df.groupby("region").agg(
    avg_resolution_time=("resolution_time_hours", "mean"),
    avg_satisfaction=("customer_satisfaction_score", "mean"),
).reset_index().sort_values("avg_resolution_time")
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
ax0 = sns.barplot(data=region_perf, x="avg_resolution_time", y="region", palette="crest", ax=axes[0])
axes[0].set_title("Avg Resolution Time by Region")
axes[0].set_xlabel("Hours")
axes[0].set_ylabel("Region")
for container in ax0.containers:
    ax0.bar_label(container, fmt=lambda x: f"{x:.1f}", fontsize=9, padding=3)

ax1 = sns.barplot(data=region_perf, x="avg_satisfaction", y="region", palette="rocket", ax=axes[1])
axes[1].set_title("Avg Satisfaction Score by Region")
axes[1].set_xlabel("Score (1-5)")
axes[1].set_ylabel("")
for container in ax1.containers:
    ax1.bar_label(container, fmt=lambda x: f"{x:.2f}", fontsize=9, padding=3)

plt.tight_layout()
plt.savefig("images/M6/plot4_region_comparison.png", dpi=150)
plt.show()
```



Milestone 4 - Week 5

Module 7

Dashboard/Report Preparation

- Combine visual insights into an interactive Power BI or Streamlit dashboard
- Organize visualizations into performance KPIs: Average Resolution Time, Most Frequent Categories, Cluster Similarity Index, Top Performing Regions

Page 1: Executive Overview



Top numbers: 200K tickets total, ~120 hrs to resolve, satisfaction and SLA numbers look off/need checking.

Ticket Type: Requests are the biggest chunk (40%), Incidents and Problems split the rest evenly.

Trend Over Time: Ticket volume goes up and down each month, no clear pattern.

SLA Compliance: Exactly half of tickets breached SLA, half didn't.

Satisfaction by Channel: Every channel (chat, email, phone, etc.) scores the same — 3.0. No difference between them.

Ticket Status: Tickets are spread evenly across all statuses (Open, Closed, In Progress, etc.) — about 20% each.

Priority Level: High, Low, Medium, Urgent all have the exact same number of tickets (50K each).

Page 2: Category Performance



Ticket Volume by Category: All categories (Account Suspension, Bug Report, Data Sync, etc.) have exactly 20K tickets each — perfectly even.

Avg Resolution Time by Category: Every category takes about the same time, ~120–121 hours, no real difference between them.

Avg Resolution Time by Priority: High, Low, Medium, Urgent all take ~120–121 hours too — priority doesn't speed things up.

Queue Team Performance: Account Management, Billing Support, and Technical Support handle more tickets (~40K each) than the other four teams (~20K each), but status mix (Open, Closed, etc.) is evenly split within each.

Product-wise Support Issues: Every product (Billing System, Cloud Storage, CRM, etc.) has the same 20K tickets — no product stands out as more problematic.

Page 3: Cluster Insights



Ticket Distribution Across Clusters: Cluster 0 is the biggest (~50% of all tickets), followed by Cluster 3 (20%), Cluster 2 (20%), and Cluster 1 (10%) — clusters are unevenly sized.

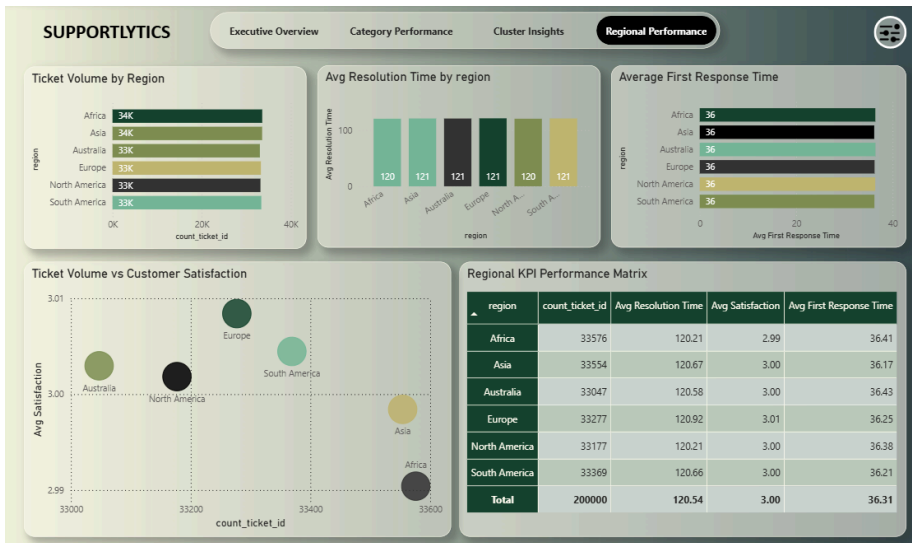
Avg Similarity Score by Cluster: Cluster 2 has the highest similarity (1.00, meaning tickets inside it are very alike), while Cluster 0 has the lowest (0.45, meaning it's the most mixed/diverse group).

Cluster Priority Distribution: Cluster 0 has the most tickets split evenly across all 4 priorities (25K each); Cluster 1 is smaller but still evenly split (10K each).

Cluster-wise Ticket Category Analysis: Cluster 0 has way more tickets across every category (~10K each) compared to Clusters 1, 2, and 3, which are much smaller.

Cluster Queue Performance: Cluster 0 dominates every team's workload (Account Management, Technical Support, Billing Support, etc.), while Clusters 1–3 contribute smaller, similar-sized shares.

Page 4: Regional Performance



Avg Resolution Time by Region: Resolution time is nearly identical everywhere (~120–121 hours), regardless of

region.

Average First Response Time: All regions respond in about the same time too (~36 hours) — flat across the board.

Ticket Volume vs Customer Satisfaction: Satisfaction scores barely vary (2.99–3.01) across regions, and there's no real link between ticket volume and satisfaction — it's basically random scatter.

Regional KPI Matrix: Confirms everything above in one table — every region is nearly identical on volume, resolution time, satisfaction, and response time, with only tiny decimal-level differences.